CIPHERMESH

SELF-INTERESTED AGENT UTILITY OPTIMIZATION FOR E-COMMERCE

Technical Implementation Report

Student Kumar Waibhav Akshat

Registration ID RA2212704010027

Programme Integrated M.Tech - CSE (Data Science)

Section AG1

Project module CipherMesh Utility Optimization

Implementation version 0.4.0

Date 12 October 2026

DELIVERABLE AT A GLANCE

A validated utility equation, a constraint-first search algorithm, ten reproducible e-commerce options, an independent optimality check, robust boundary handling, automated tests, and machine-readable evidence.

LOCAL-FIRST • DETERMINISTIC • EXPLAINABLE • TESTABLE

Contents

Executive summary What was built and the verified result

1. Objective and scope Requirement mapping and explicit scope boundaries

2. Architecture Validation, constraints, scoring, selection, verification

3. Scenario and constraints Order assumptions and hard feasibility gates

4. Mathematical utility model Normalized scores, weights, objective, tie-breaks

5. Search algorithm Single-pass argmax and exhaustive verification

6. Ten-option evaluation Ranking, result evidence, why the winner is feasible

7. Robustness and constraint boundaries Invalid data, thresholds, duplicates, empty/no-feasible cases

8. Reproduce the execution Install, run, export evidence, expected result

9. Automated verification Tests and coverage summary

10. Project component map Files and responsibilities

11. Limitations and next steps Production considerations

Conclusion Summary of engineering contribution

PRIMARY REQUIREMENT

Among ten complex offers, choose the highest-utility option that also satisfies every binding constraint. A high utility score cannot override stock, budget, or service eligibility.

Executive summary

This module implements a self-interested decision agent that chooses the best supplier offer for an e-commerce order. It does not simply select the cheapest item or the highest unfiltered score. It combines product quality, landed cost, delivery speed, reliability, and return risk, then applies hard constraints for budget, stock, activation, and destination support.

Ten deliberately varied offers are evaluated. Six pass all hard constraints and four are rejected. The winner is ECO-08 / NorthPeak, with a landed unit cost of $131.00 and utility 71.2388/100. The selection is compared with an independent exhaustive reference implementation, which selects the same candidate.

KEY CONSTRAINT SAFETY RESULT

SwiftSupply scores approximately 73.33 utility points, higher than NorthPeak, but it only has 80 units in stock when the order requires 100. CipherMesh rejects it and chooses the best feasible option instead of making an impossible allocation.

1. Objective and scope

Objective: formulate and execute mathematical utility scoring mechanics within a self-interested agent, resolve a resource-constrained allocation, and produce evidence that the mathematically best feasible decision was chosen from ten options.

1.1 Deliverables implemented

- Validated offer, weight, constraint, score-breakdown, candidate-evaluation, and result models.
- Five-factor utility equation with normalized scores and weights that must sum to 1.0.
- Constraint-first selection using a one-pass incumbent search and deterministic tie-breaks.
- Ten-option reproducible supplier scenario with utility ranking and explanatory rejection reasons.
- Independent exhaustive reference check demonstrating that the chosen candidate is the best feasible offer.
- Tests for malformed values, non-finite numbers, duplicate IDs, thresholds, ties, and empty/no-feasible inputs.
- Terminal, CSV, and JSON evidence plus this report.

1.2 Scope boundary

The offers and metrics are synthetic data for an academic demonstration. This module does not contact sellers, reserve stock, place orders, or initiate payments. The weights are explicit policy parameters and should be calibrated with business stakeholders before real deployment.

2. Architecture

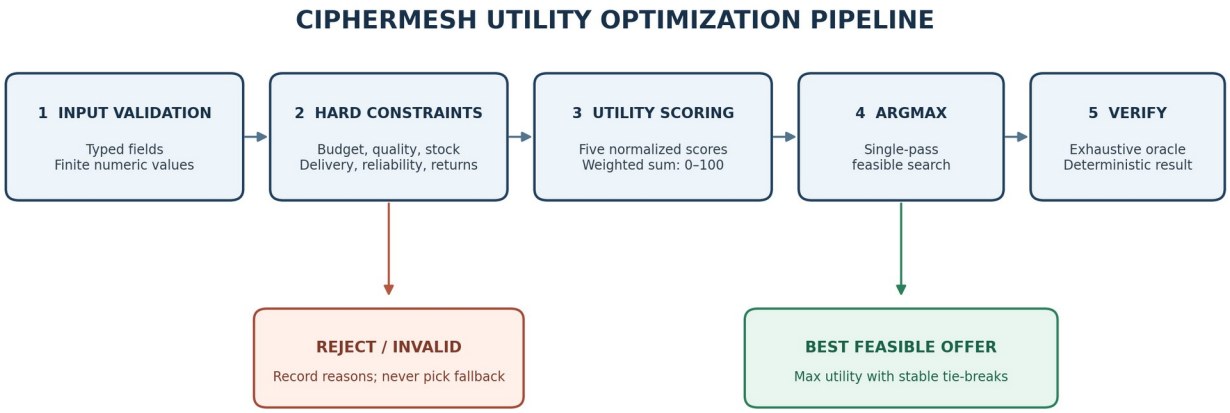


Figure 1. Constraint-first evaluation and selection pipeline.

Validate: Parse each record into a typed Offer. Invalid inputs are isolated as INVALID_DATA so one malformed seller record does not crash evaluation of the others.

Protect identity integrity: If valid rows share the same option_id, every duplicate instance is rejected. The agent never arbitrarily selects one ambiguous record.

Score: Calculate landed cost plus five normalized utility component scores.

Enforce hard constraints: Mark candidates INFEASIBLE if any non-negotiable condition fails. An infeasible option cannot win even if its score is higher.

Optimize and verify: Select with a one-pass incumbent; optionally rank all feasible offers and compare against a separate exhaustive oracle.

Report: Return selected offer, scores, feasible ranking, reason codes, counts, and optional JSON/CSV outputs.

DESIGN PRINCIPLE

Eligibility is decided before preference optimization. Utility ranks eligible options; it does not compensate for a hard constraint violation.

3. Scenario and constraints

The agent is selecting a supplier to fulfil a 100-unit order for one delivery destination. Landed cost is item price plus per-unit shipping. The candidate must satisfy every hard constraint shown below.

Constraint	Default	Meaning
Maximum landed cost per unit	$145.00	Item price + shipping must not exceed budget
Required quantity	100 units	Available stock must satisfy the complete order
Minimum quality	4.0 / 5.0	Reject below-standard product offers
Maximum delivery	5 days	Reject offers outside target delivery window
Minimum reliability	95%	Reject unreliable fulfilment profiles
Maximum return rate	6%	Reject excessive return risk
Seller status	Active	Inactive offers cannot be selected
Destination coverage	Supported	Seller must serve the target region

Thresholds are inclusive: a landed cost of exactly $145.00, quality of exactly 4.0, reliability of exactly 95%, return rate of exactly 6%, or stock of exactly 100 units can pass that respective gate if all other constraints pass.

If no candidate satisfies the complete constraint set, the optimizer returns NO_FEASIBLE_OPTION. It does not change the budget, relax quality, or select an unacceptable fallback without an explicit new policy.

4. Mathematical utility model

4.1 Notation

- P_i: item price per unit; S_i: shipping per unit; $L_i = P_i + S_i$: landed unit cost.
- B: maximum landed cost budget; Q_i: quality rating on a 0-5 scale.
- d_i: delivery days; Dmax: maximum accepted delivery days.
- R_i: seller reliability percentage; t_i: expected return-rate percentage.
- Tscale: return-rate score reference (10% in the default profile).
- clamp(x, 0, 100): bounds a score to the interval 0 through 100.

4.2 Normalized component scores

Dimension	Equation
Quality	$QScore_i = clamp(100 \times Q_i / 5, 0, 100)$
Cost	$CScore_i = clamp(100 \times (1 - L_i / B), 0, 100)$
Delivery	$DScore_i = clamp(100 \times (Dmax - d_i) / (Dmax - 1), 0, 100)$, when Dmax > 1
Delivery edge case	When Dmax = 1: one-day delivery scores 100; otherwise scores 0 (avoids divide-by-zero).
Reliability	$RScore_i = clamp(R_i, 0, 100)$
Return risk	$TScore_i = clamp(100 \times (1 - t_i / Tscale), 0, 100)$

A lower landed cost scores better. Faster delivery and lower return rate score better. The cost and delivery scores are preference measurements; budget and delivery limits are enforced separately as hard constraints.

4.3 Weighted utility equation

$$U_i = 0.30 \times QScore_i + 0.25 \times CScore_i + 0.20 \times DScore_i + 0.15 \times RScore_i + 0.10 \times TScore_i$$

Factor	Weight	Rationale
Product quality	0.30	Customer outcome and product suitability
Landed cost	0.25	Commercial efficiency without overriding budget gate
Delivery speed	0.20	Customer convenience and waiting time
Reliability	0.15	Predictable fulfilment and fewer service failures
Return risk	0.10	Reverse-logistics and customer-friction exposure
TOTAL	1.00	Validated at configuration time

Utility is an explainable preference score, not a probability. A score of 71.2388 does not imply a 71.2% chance of success; it means the configured weighted evaluator assigns that relative value on the 0-100 scale.

4.4 Objective and tie-breaks

F = { i : offer i passes every hard constraint }
Selected offer i* = arg max(U_i), for i in F
Tie-break order: utility descending → landed cost ascending → delivery days ascending → option ID ascending

5. Search algorithm

The production selector uses a single-pass incumbent search, rather than enumerating every possible permutation of offers.

```
best = NONE
for each validated offer:
   calculate component scores and utility
   if any hard constraint fails:
      record INFEASIBLE with every applicable reason
      continue
   if best is NONE or candidate wins by utility/tie-break order:
      best = candidate
return best or NO_FEASIBLE_OPTION
```

5.1 Complexity

Operation	Complexity	Explanation
Winner selection	O(n)	Each offer is evaluated once; the current best feasible option is maintained as an incumbent.
Evaluation storage	O(n)	One result record is retained for each input so that failures remain explainable.
Optional feasible ranking	O(k log k)	Sort the k feasible candidates for the human-readable ranking, where $k \leq n$.

5.2 Independent optimality verification

For the demo, a separate exhaustive reference routine independently filters feasible candidates and orders them by the same utility/tie-break definition. The runtime result is compared with this reference. The equality check is recorded as optimality_verified = true. This checks the chosen result for the ten-item fixture; it does not claim that no unobserved seller in a real marketplace can be better.

MATHEMATICAL GUARANTEE FOR THE FIXTURE

Every valid input offer is scored, each feasible candidate is considered by the one-pass selection loop, and the best incumbent is replaced whenever a strictly better candidate under the total ordering appears. Therefore, after the final input, the incumbent is the highest-utility feasible candidate in the provided finite set.

6. Ten-option evaluation

The reproducible fixture in ciphermesh/optimization/scenarios.py varies item price, shipping, quality, delivery, reliability, return rate, stock, seller activation, and destination support. Scores are retained for rejected valid offers to make the constraint decision auditable, but only FEASIBLE candidates enter the ranking.

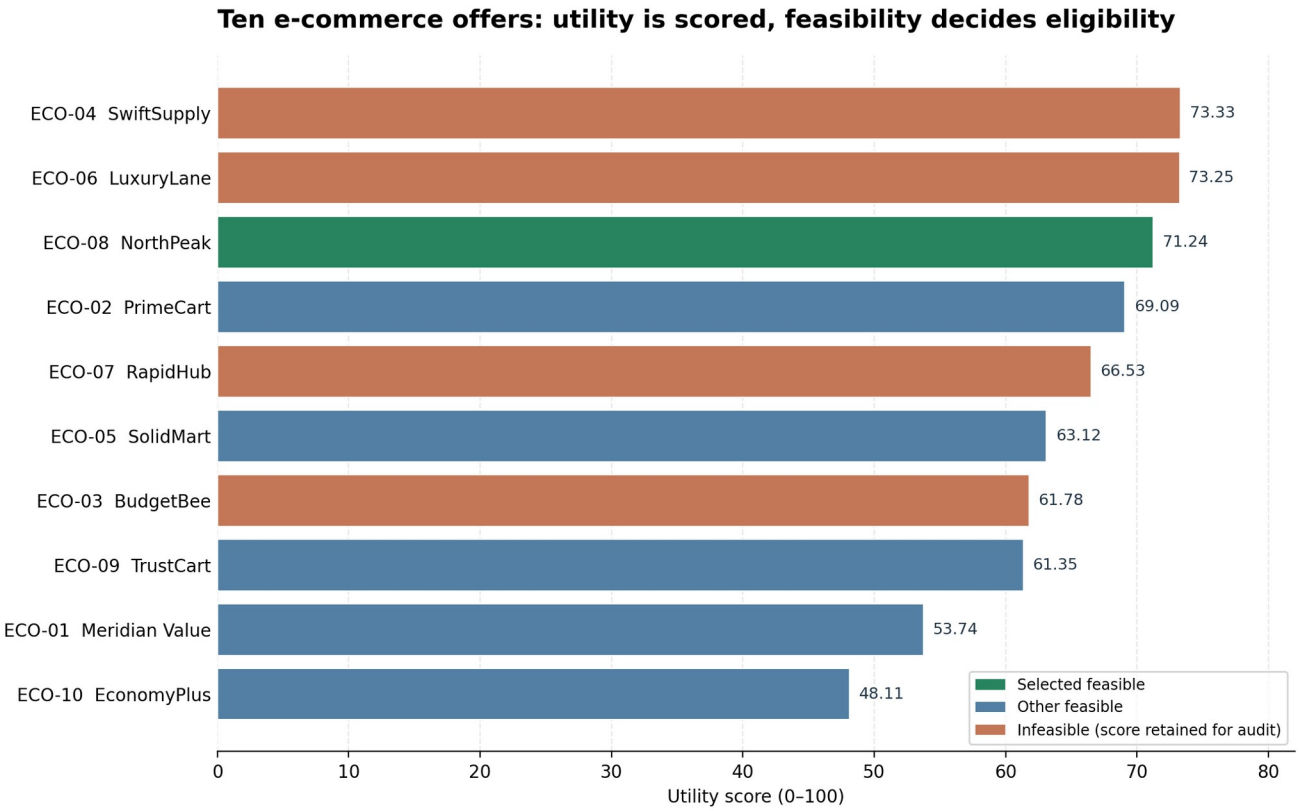


Figure 2. Utility scores for all ten offers. Green = selected feasible; blue = other feasible; orange = infeasible, score shown for audit only.

OPTIMIZER DECISION

Selected: NorthPeak (ECO-08) | Landed unit cost: $131.00 | Utility: 71.2388/100 | Feasible: 6/10 | Rejected: 4/10

6.1 Feasible ranking

Rank	ID	Seller	Landed	Utility
1	ECO-08	NorthPeak	$131.00	71.2388
2	ECO-02	PrimeCart	$130.00	69.09
3	ECO-05	SolidMart	$122.00	63.12
4	ECO-09	TrustCart	$127.00	61.35
5	ECO-01	Meridian Value	$121.00	53.74
6	ECO-10	EconomyPlus	$111.00	48.11

6.2 Full candidate-level evidence

The following table compresses operational attributes into readable columns. All ten options are evaluated, and every rejected row carries an explicit reason.

ID / Seller	Landed	Quality / Delivery	Reliability / Return	Stock	Utility / decision
ECO-01 Meridian Value	$121.00	4.2/5; 4d	96.0%; 5.0% returns	250	53.74 / FEASIBLE FEASIBLE - all hard constraints pass
ECO-02 PrimeCart	$130.00	4.8/5; 2d	98.0%; 2.0% returns	800	69.09 / FEASIBLE FEASIBLE - all hard constraints pass
ECO-03 BudgetBee	$108.00	3.7/5; 3d	98.0%; 1.5% returns	400	61.78 / INFEASIBLE Quality below min; Destination unsupported
ECO-04 SwiftSupply	$145.00	4.9/5; 1d	99.5%; 1.0% returns	80	73.33 / INFEASIBLE Stock 80 < 100
ECO-05 SolidMart	$122.00	4.6/5; 3d	97.0%; 3.0% returns	500	63.12 / FEASIBLE FEASIBLE - all hard constraints pass
ECO-06 LuxuryLane	$148.00	4.9/5; 1d	99.0%; 1.0% returns	900	73.25 / INFEASIBLE Over budget
ECO-07 RapidHub	$117.00	4.1/5; 2d	94.0%; 2.0% returns	350	66.53 / INFEASIBLE Reliability below min
ECO-08 NorthPeak	$131.00	4.7/5; 1d	97.5%; 4.0% returns	180	71.24 / SELECTED FEASIBLE - all hard constraints pass
ECO-09 TrustCart	$127.00	4.9/5; 4d	99.0%; 1.0% returns	120	61.35 / FEASIBLE FEASIBLE - all hard constraints pass
ECO-10 EconomyPlus	$111.00	4.0/5; 5d	95.0%; 6.0% returns	100	48.11 / FEASIBLE FEASIBLE - all hard constraints pass

7. Robustness and constraint boundaries

The optimization module uses typed input validation, finite numeric bounds, explicit feasibility reasons, and explicit result statuses. It is designed to avoid silent constraint relaxation or arbitrary winner selection.

Boundary / error	Defined behavior
Negative item or shipping price	INVALID_DATA - candidate excluded
NaN or infinity in a numeric field	INVALID_DATA - no utility computed
Out-of-range quality, reliability, return rate, delivery, or stock	INVALID_DATA - typed range check fails
Unknown input field	INVALID_DATA - schema rejects extra fields
Duplicate option ID	Every row with that ID is INVALID_DATA
Exact maximum budget, exact minimum quality/reliability, exact stock	Passes that boundary if every other gate passes
One step outside a hard constraint	INFEASIBLE with reason code
Dmax = 1 day	Special-case delivery formula; no division by zero
Empty input	NO_OPTIONS
No feasible options	NO_FEASIBLE_OPTION; do not select fallback
Weight sum differs from 1.0	Global configuration rejected with validation error
Equal utility	Stable cost, delivery, then ID tie-break

7.1 Error containment philosophy

One malformed candidate is isolated so valid candidates can still compete. In contrast, invalid global policy configuration (for example, weights that do not sum to one or a zero budget) fails fast and must be corrected. When no valid candidate remains, the module reports a no-selection status; it does not loosen the business policy automatically.

8. Reproduce the execution

8.1 Requirements and install

Python 3.10 or later, Pydantic 2.6+, and pytest for the full test suite. No cloud credentials or paid service are needed. From the directory containing pyproject.toml:

```
# Windows PowerShell
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"

# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev]"
```

8.2 Run and export evidence

```
python -m ciphermesh.optimization.demo
python -m ciphermesh.optimization.demo --json-out docs/examples/utility_optimization_result.json --csv-out
docs/examples/utility_options_ranking.csv
python -m pytest -q
```

To demonstrate that constraints are not silently relaxed, run with a restrictive budget:

```
python -m ciphermesh.optimization.demo --budget 100
```

Expected result for the default fixture:

```
EVALUATED=10 | VALID=10 | FEASIBLE=6 | REJECTED=4
FEASIBLE RANKING: ECO-08 > ECO-02 > ECO-05 > ECO-09 > ECO-01 > ECO-10

OPTIMIZER DECISION
  Selected seller  : NorthPeak (ECO-08)
  Landed unit cost : $131.00
  Utility score    : 71.2388 / 100
  Oracle comparison: PASS (single-pass winner=ECO-08; exhaustive reference=ECO-08)
```

8.3 Evidence files

- docs/examples/utility_optimization_terminal_timeline.txt - full terminal table and selected option.
- docs/examples/utility_optimization_result.json - typed machine-readable result and component scores.
- docs/examples/utility_options_ranking.csv - spreadsheet-ready candidate evaluations.

9. Automated verification

TEST RESULT

53 automated tests passed in the integrated project test suite, covering the existing CipherMesh policy, gateway, TCP transport, multi-agent negotiation, and this new utility optimization module.

9.1 Utility module test coverage

- Ten-option expected result and independent exhaustive optimality verification.
- Constraint-first behavior when an infeasible option has a greater raw utility score.
- Quality, cost, delivery, reliability, and return-risk arithmetic.
- Exact thresholds and one-step-outside violations for all hard constraints.
- Malformed NaN, infinity, negative, and out-of-range inputs isolated as INVALID_DATA.
- Duplicate IDs excluded consistently; no arbitrary winner.
- Empty input and no-feasible-set statuses.
- Weight validation, deterministic ties, feasible ranking, and one-day delivery edge case.

A passing suite verifies the implemented and exercised cases. It is not a proof of production performance or correctness for unmodeled business data.

10. Project component map

File / folder	Responsibility
ciphermesh/optimization/__init__.py	Public optimization API exports.
ciphermesh/optimization/models.py	Offer schema, utility weights, constraints, scores, evaluation states, results.
ciphermesh/optimization/engine.py	Component scores, hard-constraint checks, single-pass selection, exhaustive oracle.
ciphermesh/optimization/scenarios.py	Ten deterministic synthetic offers and default constraint profile.
ciphermesh/optimization/demo.py	CLI demonstration, detailed terminal table, CSV/JSON export.
tests/test_utility_optimization.py	Mathematics, optimality, boundaries, malformed data, and tie tests.
docs/examples/	Terminal timeline, candidate CSV, and result JSON.
docs/figures/	Architecture pipeline and utility-score visualization.

11. Limitations and production next steps

- Weights are hand-specified for an academic baseline. Calibrate them with stakeholders and test sensitivity before a commercial decision uses them.
- Supplier facts (stock, price, region, reliability) are assumed inputs. A real application must validate data freshness and re-check price and stock immediately before purchase.
- The utility is linear and additive. It does not model interactions, uncertainty, minimum-order tiers, volume discounts, correlated failures, or multiple simultaneous orders.
- This is a single-order, single-winner optimizer. Shared capacity across multiple orders may require linear/integer programming or another global constrained solver.
- The exhaustive oracle checks the same finite candidate set; it cannot prove there is no better unobserved supplier.
- Floating-point values are used for readability. Financial settlement should use Decimal or integer minor units with a declared rounding policy.
- The module evaluates offers but does not make purchases. Consequential actions should require explicit authorization and a final live availability check.

Conclusion

CipherMesh now contains a transparent utility evaluator that connects a formal preference model to a safe constrained decision. In the ten-option fixture, the agent selects the highest-utility feasible offer, rejects attractive but invalid alternatives, records reasons for its decision, and verifies its selected candidate against an independent exhaustive reference. This meets the implementation objective for mathematical utility evaluation, optimization search, and constraint-boundary resilience while keeping future commercial integration clearly separated from the tested academic baseline.